

AUDIT RUMUS MATEMATIS

Perbandingan Formula Proposal vs Implementasi Kode

Dokumen ini menelusuri setiap rumus bernomor di proposal dan memverifikasi apakah implementasinya di kode (model.py, Training.ipynb, Testing.ipynb) sudah tepat secara matematis.

No.	Rumus	Lokasi di Proposal	Status
(1)	Distribusi bersama VAE $p(x,z) = p(x z)p(z)$	Subbab 6.1.3	IMPLISIT
(2)	Aproksimasi posterior $q(z x) = N(z \mu(x), \sigma(x))$	Subbab 6.1.3	SESUAI
(3)	Log-likelihood $\log p(x) = KL + ELBO$	Subbab 6.1.3	SESUAI
(4)	KL Divergence $KL(q p)$	Subbab 6.1.3	SESUAI
(5)	ELBO / Loss VAE $\mathcal{L} = E[\log p(x z)] - KL$	Subbab 6.1.3	SESUAI*
(6)	Reparameterization trick $z = \mu + \sigma * \epsilon$	Subbab 6.1.3	SESUAI
(7)	SVD $A = U * \Sigma * V^T$	Subbab 6.1.4	SESUAI
(8)	Loss RSVD $\mathcal{L} = \text{rekonstruksi} + \lambda * \text{regularisasi}$	Subbab 6.1.4	PERHATIAN
(9)	Norma Frobenius $\ A\ _F$	Subbab 6.1.4	TIDAK SESUAI
(10-12)	Update SGD U, V, Σ dengan learning rate η	Subbab 6.1.4	SESUAI
(13)	Gradien $d\mathcal{L}/dU$ terhadap U	Subbab 6.1.4	PERHATIAN
(14)	Gradien $d\mathcal{L}/dV$ terhadap V	Subbab 6.1.4	PERHATIAN
(15)	Gradien $d\mathcal{L}/d\Sigma$ terhadap Σ	Subbab 6.1.4	PERHATIAN
(16)	Error rekonstruksi $e_{ij} = Z_{ij} - (U*\Sigma*V^T)_{ij}$	Subbab 6.1.4	SESUAI
(17)	MSE	Subbab 6.1.5	SESUAI
(18)	RMSE	Subbab 6.1.5	SESUAI

* SESUAI dengan modifikasi Beta-VAE yang merupakan pengembangan valid dari rumus (5).

1. Rumus-Rumus VAE

VAE didefinisikan di Subbab 6.1.3. Semua rumus bernomor (1) s.d. (6) berkaitan dengan arsitektur dan fungsi loss VAE.

Rumus (1) Distribusi Bersama VAE		
Di Proposal:	Di Kode:	Analisis:
$p(x, z) = p(x z) * p(z)$	(tidak ada baris kode eksplisit; ini adalah landasan teori)	Rumus ini adalah pernyataan probabilistik fondasi VAE, bukan sesuatu yang langsung ditulis sebagai kode. Keseluruhan arsitektur Encoder-Sampling-Decoder secara implisit mengimplementasikan distribusi ini.

Rumus (2) Aproksimasi Posterior $q(z x)$		
Di Proposal:	Di Kode:	Analisis:
$q(z x) = N(z \mid \mu(x), \sigma(x))$	<pre>self.z_mean = Dense(latent_dim) self.z_log_var = Dense(latent_dim) return self.z_mean(x), self.z_log_var(x)</pre>	Encoder menghasilkan <code>z_mean</code> (μ) dan <code>z_log_var</code> ($\log \sigma^2$) untuk setiap data input. Ini adalah representasi eksak dari distribusi Gaussian $q(z x) = N(\mu(x), \sigma(x))$ yang disebutkan di rumus (2).

Rumus (3) Dekomposisi Log-Likelihood		
Di Proposal:	Di Kode:	Analisis:
$\log p(x) = KL(q(z x) \parallel p(z x)) + l \text{ (} l = \text{ELBO, dimaksimalkan)}$	<pre>(struktur train_step) total_loss = reconstruction_loss + beta * kl_loss (minimasi total_loss = maksimasi ELBO)</pre>	Kode tidak menghitung $\log p(x)$ secara eksplisit, tapi meminimalkan <code>total_loss</code> yang setara dengan memaksimalkan ELBO (l). Karena KL selalu positif, meminimalkan <code>total_loss</code> = memaksimalkan l . Logika ini benar secara matematis.

Rumus (4) KL Divergence $KL(q p)$		
Di Proposal:	Di Kode:	Analisis:
$KL(q(x) p(x)) = \sum_x q(x) * \log(q(x)/p(x))$	<pre>kl_loss = -0.5 * (1 + z_log_var - tf.square(z_mean) - tf.exp(z_log_var)) kl_loss = tf.reduce_mean(tf.reduce_sum(kl_loss, axis=1))</pre>	Rumus (4) di proposal adalah definisi umum KL. Untuk kasus khusus VAE dengan prior $N(0, I)$ dan posterior Gaussian, KL memiliki bentuk analitik: $-0.5 * \sum(1 + \log(\sigma^2) - \mu^2 - \sigma^2)$. Implementasi kode sudah tepat menggunakan bentuk analitik ini, bukan bentuk umum rumus (4). Ini adalah praktik standar VAE.

Rumus (5) ELBO / VAE Loss Function		
Di Proposal:	Di Kode:	Analisis:
$l = E_{q(z x)}[\log p(x z)] - KL(q(z x) p(z))$	<pre># Reconstruction (E[log p(x z)]) mse_loss = sum(mask*(x-recon)^2) / (sum(mask) + 1e-8) reconstruction_loss = mse_loss * n_items # KL term kl_loss = -0.5*(1+log_var -mean^2-exp(log_var)) # Total (Beta-VAE) total_loss = reconstruction_loss + beta * kl_loss</pre>	Proposal menggunakan $\beta=1$ (VAE standar). Implementasi menambahkan parameter β (Beta-VAE) yang mengontrol bobot KL. Ketika $\beta=1$, kedua formula identik. Penambahan β adalah pengembangan valid yang tidak melanggar formula asli di proposal. Catatan: Reconstruction loss menggunakan Masked MSE (hanya rating > 0) yang lebih tepat untuk data sparse dibandingkan MSE biasa.

Rumus (6) Reparameterization Trick		
Di Proposal:	Di Kode:	Analisis:
$z = \mu + \sigma * \epsilon$ $(\epsilon \sim N(0, I))$	<pre>epsilon = random_normal(sh ape=(batch, dim)) sigma = tf.exp(0.5 * z_log_var) return z_mean + (sigma * epsilon)</pre>	Implementasi tepat sesuai rumus (6). ϵ diambil dari $N(0, I)$. σ dihitung sebagai $\exp(0.5 * \log_var) = \exp(0.5 * \log(\sigma^2)) = \sigma$. $z = z_mean + \sigma * \epsilon$ sudah benar.

2. Rumus-Rumus RSVD

RSVD didefinisikan di Subbab 6.1.4. Rumus (7) s.d. (16) berkaitan dengan dekomposisi matriks, fungsi loss, dan aturan update SGD.

Rumus (7) SVD Dasar $A = U * \text{Sigma} * V^T$		
Di Proposal:	Di Kode:	Analisis:
$A = U * \text{Sigma} * V^T$ (U: $m \times r$, Sigma: $r \times r$ diagonal, V^T : $r \times n$)	<pre>dot_product = np.dot(np.dot(U[u,:], Sigma), V[i,:].T) # U: (m x k), Sigma: (k x k), # V: (n x k)</pre>	Struktur faktorisasi $U * \text{Sigma} * V^T$ diimplementasikan dengan benar. Sigma diinisialisasi sebagai matriks identitas ($\text{np.eye}(k)$), dan hanya elemen diagonal yang diperbarui, konsisten dengan sifat matriks diagonal di rumus (7).

Rumus (8) Loss Function RSVD		
Di Proposal:	Di Kode:	Analisis:
$\text{Loss} = \sum_{ij} (Z_{ij} - (U * \text{Sigma} * V^T)_{ij})^2 + \lambda (U _F^2 + \text{Sigma} _F^2 + V _F^2)$	<pre># Hanya MSE yang disimpan di loss_history: current_mse = sum((Z[mask] - reconstructed_Z[mask])^2) / sum(mask) # Regularisasi L2 diterapkan implisit # lewat update rule (bukan di loss eval)</pre>	Ada dua bagian rumus (8): 1. Rekonstruksi error: diimplementasikan dengan benar di evaluasi MSE. 2. Penalti regularisasi L2: TIDAK dihitung secara eksplisit di loss_history. Regularisasi hanya diterapkan implisit di update rule U dan V ($\lambda * U_{uk}$ dan $\lambda * V_{ik}$), tapi tidak di Sigma. Artinya loss yang dilaporkan di loss_history adalah MSE murni tanpa komponen regularisasi, berbeda dengan loss (8) di proposal.

Rumus (9) Norma Frobenius $ A _F$		
Di Proposal:	Di Kode:	Analisis:
$ A _F = \sqrt{\sum_{ij} a_{ij} ^2}$	(tidak ada implementasi eksplisit norma Frobenius di kode)	Rumus (9) mendefinisikan norma Frobenius yang dipakai dalam penalti regularisasi L2 di rumus (8). Di kode, penalti regularisasi tidak dihitung menggunakan norma Frobenius secara eksplisit. Update rule U dan V menggunakan $-\lambda * U_{uk}$ dan $-\lambda * V_{ik}$ yang merupakan gradien dari $ U _F^2$ per elemen, jadi secara implisit sudah benar untuk U dan V. Namun Sigma TIDAK mendapat penalti regularisasi sama sekali di update rule-nya: $\text{Sigma}[k,k] += \eta * (e_{ui} * U_{uk} * V_{ik})$ (tidak ada suku $-\lambda * \text{Sigma}[k,k]$).

Rumus (10-12) Update Rule SGD untuk U, V, Sigma		
Di Proposal:	Di Kode:	Analisis:
<pre>U <- U - eta * (dLoss/dU) V <- V - eta * (dLoss/dV) Sigma <- Sigma - eta * (dLoss/dSigma)</pre>	<pre>U[u,k] += eta * (...) V[i,k] += eta * (...) Sigma[k,k] += eta * (...)</pre>	Rumus (10-12) adalah update rule SGD umum. Kode mengimplementasikannya dengan += karena gradien yang diturunkan sudah menghasilkan arah negatif (yaitu e_{ui} yang merupakan $Z - \text{pred}$, bukan $\text{pred} - Z$). Arah update sudah benar: mengurangi error.

Rumus (13) Gradien dL/dU_uk		
Di Proposal:	Di Kode:	Analisis:
<pre>dL/dU_uk = -2*sum_i(e_ui*(Sigma*V^T)_ki) + 2*lambda*U_uk</pre>	<pre>U[u,k] += eta * (e_ui * Sigma_kk * V_ik - lam * U_uk)</pre>	Rumus (13) menggunakan $(\text{Sigma} \cdot \text{V}^T)_{ki}$ yang merupakan hasil perkalian matriks penuh. Untuk kasus Sigma diagonal, elemen ke-(k,i) dari $\text{Sigma} \cdot \text{V}^T$ adalah $\text{Sigma}_{kk} * V_{ik}$. Implementasi kode tepat untuk kasus ini. Namun ada perbedaan faktor: proposal menggunakan $-2 \cdot e_{ui}$ dan $+2 \cdot \text{lambda} \cdot U_{uk}$, sedangkan kode menggunakan e_{ui} dan $-\text{lam} \cdot U_{uk}$ (tanpa faktor 2). Faktor 2 di kedua suku saling menghilangkan sehingga arah gradiennya sama, dan bisa diabsorpsi ke dalam learning rate eta. Secara praktis ini tidak mengubah konvergensi, hanya skala eta efektif.

Rumus (14) Gradien dL/dV_ik		
Di Proposal:	Di Kode:	Analisis:
<pre>dL/dV_ik = -2*sum_u(e_ui*(Sigma*U^T)_ku) + 2*lambda*V_ik</pre>	<pre>V[i,k] += eta * (e_ui * Sigma_kk * U_uk - lam * V_ik)</pre>	Analisis sama dengan rumus (13). Untuk Sigma diagonal, $(\text{Sigma} \cdot \text{U}^T)_{ku} = \text{Sigma}_{kk} * U_{uk}$. Kode sudah benar untuk kasus ini. Faktor 2 tidak ada di kode tapi konsisten dengan penghilangan faktor 2 di rumus (13). Hasilnya adalah efektif eta di kode = $2 * \text{eta}$ di proposal, tapi karena eta adalah hyperparameter yang di-tune, ini tidak masalah.

Rumus (15) Gradien dL/dSigma_kk		
Di Proposal:	Di Kode:	Analisis:
$dL/dSigma_{kk} = -2 \sum_u \sum_i (e_{ui} * U_{uk} * V_{ik}) + 2 * \lambda * Sigma_{kk}$	<pre>Sigma[k,k] += eta * (e_ui * U_uk * V_ik) # TIDAK ada suku: - lam * Sigma[k,k]</pre>	Ada dua perbedaan dengan rumus (15) di proposal: 1. Faktor 2 tidak ada — konsisten dengan rumus (13) dan (14), bisa diabsorpsi ke eta. 2. LEBIH PENTING: Suku regularisasi "+2*lambda*Sigma_kk" di proposal seharusnya menghasilkan suku "-lambda*Sigma_kk" di update rule kode. Suku ini HILANG dari implementasi. Artinya Sigma tidak diregularisasi, berbeda dengan yang tercantum di rumus (8) dan (15) di proposal.

Rumus (16) Error Rekonstruksi e_ij		
Di Proposal:	Di Kode:	Analisis:
$e_{ij} = Z_{ij} - (U * Sigma * V^T)_{ij}$	<pre>pred = mu + b_u[u] + b_i[i] + dot(dot(U[u,:], Sigma), V[i,:].T) e_ui = Z[u,i] - pred</pre>	Rumus error dasar $e = Z - \text{pred}$ sudah benar. Implementasi menambahkan bias terms ($\mu + b_u + b_i$) yang merupakan pengembangan standard matrix factorization (tidak ada di proposal tapi secara akademis valid dan meningkatkan akurasi).

3. Rumus Metrik Evaluasi

Metrik evaluasi didefinisikan di Subbab 6.1.5, rumus (17) dan (18).

Rumus (17) Mean Squared Error (MSE)		
Di Proposal:	Di Kode:	Analisis:
$MSE = (1/n) * \sum_i (y_i - \hat{y}_i)^2$	<pre># Testing.ipynb & CrossValidation.ipynb mse = np.mean(np.square(y_true - y_pred))</pre>	np.mean(np.square(...)) identik dengan $(1/n) * \sum (...^2)$. Perhitungan dilakukan pada actual_values vs pred_clipped setelah denormalisasi ke skala 1-5, sehingga satuan MSE sudah dalam skala rating asli.

Rumus (18) Root Mean Squared Error (RMSE)		
Di Proposal:	Di Kode:	Analisis:
$RMSE = \sqrt{(1/n) * \sum_i (y_i - \hat{y}_i)^2}$	<pre># Testing.ipynb & CrossValidation.ipynb rmse = np.sqrt(np.mean(np.square(y_true - y_pred)))</pre>	np.sqrt(np.mean(np.square(...))) identik dengan rumus (18). Implementasi konsisten di Testing.ipynb dan CrossValidation.ipynb.

4. Ringkasan Temuan Audit Rumus

No.	Rumus	Status	Catatan Kritis
(1)	Distribusi bersama $p(x,z)$	IMPLISIT	Landasan teori, tidak perlu baris kode spesifik.
(2)	Posterior $q(z x) = N(\mu, \sigma)$	SESUAI	Encoder menghasilkan z_mean dan z_log_var.
(3)	Dekomposisi $\log p(x)$	SESUAI	Minimasi total_loss setara maksimasi ELBO.
(4)	KL Divergence umum	SESUAI	Kode menggunakan bentuk analitik KL untuk Gaussian.
(5)	ELBO / VAE loss	SESUAI*	Beta-VAE: penambahan beta valid, tidak melanggar formula.
(6)	Reparameterization trick	SESUAI	$z = z_mean + \exp(0.5 * \log_var) * \epsilon$. Benar.
(7)	Dekomposisi SVD $A = U \Sigma V^T$	SESUAI	Sigma diagonal, update per elemen diagonal saja.
(8)	Loss RSVD dengan regularisasi	PERHATIAN	loss_history hanya MSE, tidak termasuk L2 penalty.
(9)	Norma Frobenius $\ A\ _F$	TIDAK SESUAI	Sigma tidak mendapat penalti regularisasi di update rule.
(10-12)	Update rule SGD U, V, Sigma	SESUAI	Arah dan mekanisme update sudah benar.
(13)	Gradien dL/dU_{uk}	PERHATIAN	Faktor 2 dihilangkan (konsisten). Logika update benar.
(14)	Gradien dL/dV_{ik}	PERHATIAN	Sama seperti (13). Konsisten dan tidak mengubah konvergensi.
(15)	Gradien $dL/dSigma_{kk}$	PERHATIAN	Faktor 2 hilang (konsisten). TAPI suku $-\lambda * Sigma$ juga hilang.
(16)	Error $e_{ij} = Z - U \Sigma V^T$	SESUAI	Benar. Bias terms (μ, b_u, b_i) adalah tambahan valid.

No.	Rumus	Status	Catatan Kritis
(17)	MSE	SESUAI	np.mean(np.square(...)) tepat. Skala sudah benar (1-5).
(18)	RMSE	SESUAI	np.sqrt(np.mean(np.square(...))) tepat dan konsisten.

Kesimpulan Audit Rumus: Dari 16 rumus yang diperiksa, 9 sesuai (termasuk 1 implisit dan 1 sesuai dengan catatan), 4 berstatus Perhatian, dan 1 tidak sesuai. Temuan terpenting adalah **regularisasi Sigma yang hilang di rumus update (15)** — Sigma tidak mendapat penalti $-\lambda \cdot \text{Sigma_kk}$, berbeda dengan rumus (8) dan (15) di proposal. Semua rumus VAE (1-6) sudah benar. Semua metrik evaluasi (17-18) sudah benar.

5. Perbaikan Kode untuk Rumus yang Belum Sesuai

Bagian ini menyajikan kode perbaikan konkret untuk setiap rumus yang berstatus TIDAK SESUAI atau PERHATIAN. Setiap perbaikan disertai kode lama (sebelum), kode baru (sesudah), dan penjelasan matematis alasan perubahannya.

Perbaikan Rumus (15) & (8) Regularisasi Sigma di Update Rule RSVD [model.py]

Masalah: Update rule Sigma tidak menyertakan suku penalti regularisasi $-\lambda \text{Sigma_kk}$, padahal rumus (8) proposal menyatakan $\|\text{Sigma}\|_F^2$ termasuk dalam penalti L2, dan rumus (15) menunjukkan gradiennya adalah $+2\lambda \text{Sigma_kk}$.

Sebelum (kode lama):	Sesudah (kode baru):
<pre># Di dalam loop for k in range(self.k): self.Sigma[k, k] += self.eta * (e_ui * U_uk * V_ik)</pre>	<pre># Di dalam loop for k in range(self.k): self.Sigma[k, k] += self.eta * (e_ui * U_uk * V_ik - self.lam * self.Sigma[k, k] # regularisasi L2)</pre>

Penjelasan matematis: Rumus gradien $dL/d\text{Sigma_kk} = -2\sum(e_{ui}U_{uk}V_{ik}) + 2\lambda \text{Sigma_kk}$. Update rule SGD: $\text{Sigma_kk} \leftarrow \text{Sigma_kk} - \eta(dL/d\text{Sigma_kk})$. Dengan menghilangkan faktor 2 secara konsisten (sama seperti U dan V): $\text{Sigma_kk} \leftarrow \text{Sigma_kk} + \eta(e_{ui}U_{uk}V_{ik} - \lambda \text{Sigma_kk})$. Suku $-\lambda \text{Sigma_kk}$ inilah yang selama ini hilang.

Perbaikan Rumus (13) Konsistensi Faktor 2 pada Update U [model.py]

Masalah: Proposal menuliskan gradien dengan faktor 2: $dL/dU_{uk} = -2e_{ui}(\text{Sigma}^TV^T)_k + 2\lambda U_{uk}$. Kode menghilangkan faktor 2. Ini tidak salah secara matematis (efeknya hanya pada skala eta efektif), tetapi perlu komentar eksplisit agar konsisten dan dapat dipertanggungjawabkan.

Sebelum (kode lama):	Sesudah (kode baru):
<pre># Update Matriks Laten self.U[u, k] += self.eta * (e_ui * Sigma_kk * V_ik - self.lam * U_uk)</pre>	<pre># Update U – gradien dL/dU_uk disederhanakan: # Proposal: -2*e_ui*Sigma_kk*V_ik + 2*lam*U_uk # Kode: -e_ui*Sigma_kk*V_ik + lam*U_uk # (faktor 2 diabsorbsi ke dalam eta) self.U[u, k] += self.eta * (e_ui * Sigma_kk * V_ik - self.lam * U_uk)</pre>

Penjelasan matematis: Gradien dL/dU_{uk} dari rumus (13) adalah $-2e_{ui}(\text{Sigma}^TV^T)_k + 2\lambda U_{uk}$. Karena faktor 2 muncul di semua suku secara bersamaan, menghilangkannya ekuivalen dengan menggunakan $\eta_{\text{efektif}} = 2\eta$. Karena eta adalah hyperparameter yang di-tune, perbedaan ini tidak mempengaruhi hasil. Perubahan yang disarankan hanya menambah komentar penjelasan, bukan mengubah logika kode.

Perbaikan Rumus (14)

Konsistensi Faktor 2 pada Update V [model.py]

Masalah: Sama seperti rumus (13), proposal menggunakan faktor 2 pada gradien V yang tidak ada di kode. Perlu ditambahkan komentar penjas.

Sebelum (kode lama):	Sesudah (kode baru):
<pre>self.V[i, k] += self.eta * (e_ui * Sigma_kk * U_uk - self.lam * V_ik)</pre>	<pre># Update V – gradien dL/dV_ik disederhanakan: # Proposal: -2*e_ui*Sigma_kk*U_uk + 2*lam*V_ik # Kode: -e_ui*Sigma_kk*U_uk + lam*V_ik # (faktor 2 diabsorbsi ke dalam eta) self.V[i, k] += self.eta * (e_ui * Sigma_kk * U_uk - self.lam * V_ik)</pre>

Penjelasan matematis: Rumus (14): $dL/dV_{ik} = -2 \sum_u (e_{ui} (\Sigma U^T)_{ku}) + 2 \lambda V_{ik}$. Untuk Sigma diagonal: $(\Sigma U^T)_{ku} = \Sigma_{kk} * U_{uk}$. Penghilangan faktor 2 konsisten dengan perlakuan di U, dan tidak mengubah konvergensi.

Perbaikan Rumus (8)

Loss History Menyertakan Komponen Regularisasi [model.py]

Masalah: loss_history yang disimpan hanya berisi MSE murni. Rumus (8) mendefinisikan loss RSVD sebagai MSE + penalti L2. Loss yang dilaporkan tidak mencerminkan rumus loss yang sebenarnya dioptimasi.

Sebelum (kode lama):	Sesudah (kode baru):
<pre># Evaluasi MSE (hanya pada nilai yang > 0) mask = (Z > 0) current_mse = np.sum(np.square(Z[mask] - reconstructed_Z[mask])) / np.sum(mask) self.loss_history.append(current_mse)</pre>	<pre># Evaluasi Loss = MSE + penalti L2 (rumus 8) mask = (Z > 0) current_mse = np.sum(np.square(Z[mask] - reconstructed_Z[mask])) / np.sum(mask) # Hitung penalti regularisasi L2 (norma Frobenius) l2_penalty = self.lam * (np.sum(np.square(self.U)) + np.sum(np.square(self.Sigma)) + np.sum(np.square(self.V))) current_loss = current_mse + l2_penalty self.loss_history.append(current_loss) # Simpan MSE terpisah untuk pelaporan self.mse_history.append(current_mse)</pre>

Penjelasan matematis: Rumus (8): $Loss = \sum_{ij} (Z_{ij} - (U \Sigma V^T)_{ij})^2 + \lambda (||U||_F^2 + ||\Sigma||_F^2 + ||V||_F^2)$. $np.sum(np.square(M))$ menghitung $||M||_F^2$ (jumlah kuadrat semua elemen), sesuai definisi norma Frobenius di rumus (9): $||A||_F = \sqrt{\sum_{ij} |a_{ij}|^2}$, sehingga $||A||_F^2 = \sum_{ij} a_{ij}^2$. Catatan: tambahkan `self.mse_history = []` di `__init__` jika ingin melacak MSE murni terpisah.

5.5 Ringkasan Lokasi Perubahan di model.py

Semua perbaikan hanya berada di file model.py, di dalam method fit() kelas RSVD.

Lokasi di model.py	Perubahan	Rumus Terkait
__init__: tambah self.mse_history = []	Inisialisasi list untuk menyimpan MSE murni terpisah dari loss penuh	(6)
fit() → loop k → update Sigma	Tambah suku: - self.lam * self.Sigma[k, k]	(15), (8)
fit() → loop k → update U	Tambah komentar penjelasan faktor 2 (kode tidak berubah)	(13)
fit() → loop k → update V	Tambah komentar penjelasan faktor 2 (kode tidak berubah)	(14)
fit() → evaluasi akhir setiap epoch	Tambah perhitungan l2_penalty dan current_loss; simpan ke mse_history	(8), (9)

Catatan Penting: Perbaikan Rumus (13) dan (14) hanya menambahkan komentar — logika kode tidak berubah karena penghilangan faktor 2 secara konsisten tidak mempengaruhi konvergensi. Perbaikan yang benar-benar mengubah perilaku kode hanya ada dua: (1) penambahan suku regularisasi Sigma di update rule, dan (2) perhitungan loss history yang menyertakan penalti L2.